

IES FRANCISCO DE GOYA – LA ELIPA DEPARTAMENTO DE INFORMÁTICA				Dirección del Área Territorial de Madrid Capital CONSEJERÍA DE EDUCACIÓN, JUVENTUD Y DEPORTE  Comunidad de Madrid
Titulación:	DAM/DAW	Curso:	2025/2026	
Módulo:	Bases de Datos	Fecha:	Mayo 2026	
Examen:	UT7: Variante 3 - Logs y Finanzas	Duración:	60 min	
Nombre:		Calificación:		
Apellidos:				
RAs:	RA5: Desarrolla procedimientos almacenados evaluando y utilizando las sentencias del lenguaje incorporado en el sistema gestor de bases de datos.			

Instrucciones

- Los ejercicios deben resolverse utilizando sintaxis estándar de MySQL.
- Se valorará la limpieza del código, el uso de delimitadores y la correcta definición de tipos.
- La base de datos de referencia para todos los ejercicios es **Sakila**.

Ejercicio 1 (4 puntos) – (RA5-CE.g) Procedimiento con bucle

Crea un procedimiento almacenado llamado `p_insert_log_entries` que genere entradas de log en una tabla de prueba.

- Recibe: `p_message`, `p_iterations` (IN) y `p_last_id` (OUT).
- Debe usar un bucle **WHILE** para insertar exactamente el número de entradas solicitado.
- Si la tabla `test_log` no existe, debe crearla dentro del procedimiento (ID autoincremental, mensaje VARCHAR(255), timestamp).
- Cada mensaje debe incluir el número de iteración actual usando `CONCAT`.
- Devuelve en el parámetro OUT el ID de la última entrada insertada usando `LAST_INSERT_ID()`.

Ejemplo de ejecución:

```
CALL p_insert_log_entries('Test message', 3, @ultimo_id);  
SELECT @ultimo_id;
```

Resultado esperado: Se insertarán 3 filas en la tabla `test_log` con mensajes como "Test message - Iteracion 0". El `SELECT` mostrará el ID de la última entrada insertada.

Pista: Usa `CREATE TABLE IF NOT EXISTS` para asegurar que la tabla existe. El bucle `WHILE` debe iterar desde 0 hasta `p_iterations - 1`.

Ejercicio 2 (3 puntos) – (RA5-CE.g) Trigger de Control de Pagos

Implementa un trigger llamado `tr_check_payment_update` en la tabla `payment` que actúe antes de actualizar un registro.

- Impide que el importe (`amount`) de un pago sea reducido respecto al valor original.
- Impide también cambios de importe que sean menores de 1 euro (excepto si el importe no cambia).
- Los mensajes de error deben ser claros y diferenciar ambos casos.

Ejemplo de ejecución:

```
UPDATE payment SET amount = 0.50 WHERE payment_id = 2;
```

Resultado esperado: ERROR 1644 (45000): Error: Cambios de importe deben ser de al menos 1 euro.

Pista: Usa `NEW.amount < OLD.amount` para detectar reducciones. Para el cambio mínimo de 1 euro, usa `ABS(NEW.amount - OLD.amount) < 1.00 AND NEW.amount <> OLD.amount`.

Ejercicio 3 (3 puntos) – (RA5-CE.g) Función de Ingresos

Crea una función almacenada llamada `f_get_store_revenue` que calcule los ingresos totales de una tienda específica.

- Recibe el ID de una tienda (`p_store_id`) y devuelve un `DECIMAL(10,2)`.
- Calcula y devuelve el total de ingresos (`SUM(amount)`) generados por todos los empleados de esa tienda.
- Realiza un `JOIN` entre las tablas `payment` y `staff` para filtrar por tienda.

Ejemplo de ejecución:

```
SELECT f_get_store_revenue(1) as ingresos_tienda_1;
```

Resultado esperado: Un valor decimal con los ingresos totales de la tienda 1 (aproximadamente **33489.47** según los datos de Sakila).

Nota: Esta función es NOT DETERMINISTIC porque los datos pueden cambiar. Debes especificar `READS SQL DATA` en su definición. Usa `IFNULL(v_total, 0.00)` para manejar casos donde no haya pagos.